

Instituto Tecnológico de San Juan del Río



Ingeniería en Sistemas Computacionales

Tópicos de Ciberseguridad

Reconstrucción del código fuente (Ghidra)

Compile and compare the new executable with its original

P R E S E N T A N:

Roberto Joshua García Solís 22590410

Oscar Gabriel Rogelio Jiménez Hernández 22590332

Carrera: Ingeniería en Sistemas Computacionales (ISC)

Periodo: Agosto – Diciembre 2026

1. Contexto

Este documento corresponde al rubro R004 de la actividad "Reconstrucción del código fuente (Ghidra)", de la materia Tópicos de Ciberseguridad. Presenta el código fuente reconstruido a partir del análisis documentado en el rubro R003, su recompilación con TCC (Tiny C Compiler), y la comparación de resultados contra el ejecutable original.

Nota: por restricción de la actividad, no se entregan archivos ejecutables (.elf ni binarios) en ningún caso; únicamente código fuente, evidencia de salida (JSON) y evidencia de la comparación (hashes, tamaños, desensamblado).

2. Código fuente original

Código fuente del programa tal como fue compilado originalmente con TCC (archivo original/reporte_temperaturas.c en este mismo directorio):

```
/*
 * reporte_temperaturas.c
 *
 * Programa de ejemplo para la actividad "Reconstruccion del codigo
 * fuente (Ghidra)". Simula las lecturas de temperatura de una estacion
 * meteorologica durante una semana, calcula estadisticas basicas
 * (minimo, maximo, promedio y numero de dias con alerta de calor) y
 * escribe un reporte en formato JSON a disco.
 *
 * Autor original: estudiante (122590410@sjuanrio.tecnm.mx)
 * Compilador objetivo: TCC (Tiny C Compiler)
 */

#include <stdio.h>

#define NUM_DIAS 7
#define UMBRAL_ALERTA 30.0

typedef struct {
    float minimo;
    float maximo;
    float promedio;
    int dias_alerta;
} Estadisticas;

/* Datos de entrada "fijos" para que el ejercicio sea reproducible */
static float lecturas[NUM_DIAS] = {
    22.5f, 25.0f, 31.2f, 28.7f, 33.1f, 19.8f, 27.4f
};

const char *dia_nombre(int indice) {
    static const char *nombres[NUM_DIAS] = {
        "Lunes", "Martes", "Miercoles", "Jueves",
        "Viernes", "Sabado", "Domingo"
    };
    if (indice < 0 || indice >= NUM_DIAS) {
        return "Desconocido";
    }
    return nombres[indice];
}

Estadisticas calcular_estadisticas(float datos[], int n) {
    Estadisticas resultado;
    float suma = 0.0f;
    int i;
```

```

    resultado.minimo = datos[0];
    resultado.maximo = datos[0];
    resultado.dias_alerta = 0;

    for (i = 0; i < n; i++) {
        float valor = datos[i];

        if (valor < resultado.minimo) {
            resultado.minimo = valor;
        }
        if (valor > resultado.maximo) {
            resultado.maximo = valor;
        }
        if (valor >= UMBRAL_ALERTA) {
            resultado.dias_alerta = resultado.dias_alerta + 1;
        }

        suma = suma + valor;
    }

    resultado.promedio = suma / (float)n;
    return resultado;
}

int escribir_reporte_json(const char *ruta, Estadisticas stats, float datos[], int n) {
    FILE *archivo;
    int i;

    archivo = fopen(ruta, "w");
    if (archivo == NULL) {
        printf("Error: no se pudo crear el archivo %s\n", ruta);
        return 1;
    }

    fprintf(archivo, "{\n");
    fprintf(archivo, "  \"estacion\": \"Central\",\n");
    fprintf(archivo, "  \"lecturas\": [\n");

    for (i = 0; i < n; i++) {
        fprintf(archivo, "    { \"dia\": \"%s\", \"temperatura\": %.1f },\n",
            dia_nombre(i), datos[i]);
        if (i < n - 1) {
            fprintf(archivo, ",\n");
        }
        fprintf(archivo, "\n");
    }

    fprintf(archivo, "  ],\n");
    fprintf(archivo, "  \"estadisticas\": {\n");
    fprintf(archivo, "    \"minimo\": %.1f,\n", stats.minimo);
    fprintf(archivo, "    \"maximo\": %.1f,\n", stats.maximo);
    fprintf(archivo, "    \"promedio\": %.2f,\n", stats.promedio);
    fprintf(archivo, "    \"dias_alerta_calor\": %d\n", stats.dias_alerta);
    fprintf(archivo, "  }\n");
    fprintf(archivo, "}\n");

    fclose(archivo);
    return 0;
}

int main(void) {
    Estadisticas stats;
    int resultado;

    printf("Generando reporte de temperaturas...\n");

```

```
stats = calcular_estadisticas(lecturas, NUM_DIAS);
resultado = escribir_reporte_json("reporte.json", stats, lecturas, NUM_DIAS);

if (resultado == 0) {
    printf("Reporte generado exitosamente en reporte.json\n");
    printf("Minimo: %.1f, Maximo: %.1f, Promedio: %.2f, Dias de alerta: %d\n",
        stats.minimo, stats.maximo, stats.promedio, stats.dias_alerta);
}

return resultado;
}
```

3. Código fuente reconstruido

Código fuente reconstruido a partir del análisis en Ghidra documentado en R003 (pseudo-código decompilado, cadenas de texto y constantes numéricas recuperadas). Archivo reconstruido/reporte_temperaturas_reconstruido.c en este mismo directorio:

```
/*
 * reporte_temperaturas_reconstruido.c
 *
 * Codigo fuente RECONSTRUIDO a partir del analisis del ejecutable
 * "reporte_temperaturas" (compilado originalmente con TCC) usando
 * ingenieria inversa con Ghidra (modo headless, decompilador nativo).
 *
 * Este archivo NO fue copiado del original: se reconstruyo leyendo
 * el pseudo-codigo C generado por el decompilador de Ghidra para cada
 * funcion, cruzandolo con:
 * - La tabla de cadenas de texto del binario (comando `strings`),
 *   que revela los literales usados en printf/fprintf.
 * - El volcado hexadecimal de la seccion de datos (.rodata/.data)
 *   para recuperar las 7 constantes flotantes del arreglo de
 *   lecturas y el valor del umbral de alerta.
 * - Los nombres de simbolos dinamicos importados (fopen, fprintf,
 *   fclose, printf, memmove) que Ghidra si conserva aun en un
 *   binario "stripped", porque provienen de la tabla de simbolos
 *   dinamicos (.dynsym) usada para la resolucio de la libc.
 *
 * Ver docs/proceso_reconstruccion.md para el detalle paso a paso.
 */

#include <stdio.h>

#define NUM_DIAS 7
#define UMBRAL_ALERTA 30.0

/*
 * Ghidra decompilo la funcion de estadisticas devolviendo un bloque
 * "undefined1 [16]" (16 bytes sin tipo): la firma real de la funcion
 * en ensamblador usa el registro XMM para regresar 4 valores de 4
 * bytes empaquetados (min, max, promedio, dias_alerta). Eso equivale
 * a una estructura de 4 campos de 4 bytes cada uno, tal como se
 * reconstruye aqui.
 */
typedef struct {
    float minimo;
    float maximo;
    float promedio;
    int dias_alerta;
} Estadisticas;

/* Recuperado del volcado de memoria de datos (offset 0xa84 del binario) */
static float lecturas[NUM_DIAS] = {
    22.5f, 25.0f, 31.2f, 28.7f, 33.1f, 19.8f, 27.4f
};

/*
 * Ghidra nombro esta funcion FUN_00400405. Se identifico su proposito
 * por el arreglo de punteros a cadenas (PTR_s_Lunes_00600aa0) y por
 * el string de respaldo "Desconocido" usado cuando el indice esta
 * fuera de rango.
 */
const char *dia_nombre(int indice) {
    static const char *nombres[NUM_DIAS] = {
        "Lunes", "Martes", "Miercoles", "Jueves",
        "Viernes", "Sabado", "Domingo"
    };
    if (indice < 0 || indice >= NUM_DIAS) {
```

```

        return "Desconocido";
    }
    return nombres[indice];
}

/*
 * Corresponde a FUN_00400451 en el listado de Ghidra. El pseudo-codigo
 * mostraba un ciclo que actualiza minimo/maximo, compara contra una
 * constante de doble precision (DAT_00600b20, identificada como 30.0
 * en el volcado de datos) y acumula una suma para el promedio.
 */
Estadisticas calcular_estadisticas(float datos[], int n) {
    Estadisticas resultado;
    float suma = 0.0f;
    int i;

    resultado.minimo = datos[0];
    resultado.maximo = datos[0];
    resultado.dias_alerta = 0;

    for (i = 0; i < n; i++) {
        float valor = datos[i];

        if (valor < resultado.minimo) {
            resultado.minimo = valor;
        }
        if (resultado.maximo < valor) {
            resultado.maximo = valor;
        }
        if (UMBRAL_ALERTA <= (double)valor) {
            resultado.dias_alerta = resultado.dias_alerta + 1;
        }

        suma = suma + valor;
    }

    resultado.promedio = suma / (float)n;
    return resultado;
}

/*
 * Corresponde a FUN_00400576. Los literales de formato JSON se
 * recuperaron con `strings -t x` sobre el ejecutable original
 * (por ejemplo: '{ "dia": "%s", "temperatura": %.1f }').
 */
int escribir_reporte_json(const char *ruta, Estadisticas stats, float datos[], int n) {
    FILE *archivo;
    int i;

    archivo = fopen(ruta, "w");
    if (archivo == NULL) {
        printf("Error: no se pudo crear el archivo %s\n", ruta);
        return 1;
    }

    fprintf(archivo, "{\n");
    fprintf(archivo, "    \"estacion\": \"Central\",\n");
    fprintf(archivo, "    \"lecturas\": [\n");

    for (i = 0; i < n; i++) {
        fprintf(archivo, "        { \"dia\": \"%s\", \"temperatura\": %.1f },",
            dia_nombre(i), datos[i]);
        if (i < n - 1) {
            fprintf(archivo, ",");
        }
    }
    fprintf(archivo, "\n");
}

```

```

    }

    fprintf(archivo, "  ],\n");
    fprintf(archivo, "  \"estadisticas\": {\n");
    fprintf(archivo, "    \"minimo\": %.1f,\n", stats.minimo);
    fprintf(archivo, "    \"maximo\": %.1f,\n", stats.maximo);
    fprintf(archivo, "    \"promedio\": %.2f,\n", stats.promedio);
    fprintf(archivo, "    \"dias_alerta_calor\": %d\n", stats.dias_alerta);
    fprintf(archivo, "  }\n");
    fprintf(archivo, "}\n");

    fclose(archivo);
    return 0;
}

/*
 * La funcion main de Ghidra mostraba el retorno de 16 bytes de
 * calcular_estadisticas siendo copiado con memmove a variables
 * locales antes de invocar escribir_reporte_json: asi se identifico
 * que ambas funciones comparten la misma estructura de 4 campos.
 */
int main(void) {
    Estadisticas stats;
    int resultado;

    printf("Generando reporte de temperaturas...\n");

    stats = calcular_estadisticas(lecturas, NUM_DIAS);
    resultado = escribir_reporte_json("reporte.json", stats, lecturas, NUM_DIAS);

    if (resultado == 0) {
        printf("Reporte generado exitosamente en reporte.json\n");
        printf("Minimo: %.1f, Maximo: %.1f, Promedio: %.2f, Dias de alerta: %d\n",
            stats.minimo, stats.maximo, stats.promedio, stats.dias_alerta);
    }

    return resultado;
}

```

4. Compilación

Ambos archivos se compilaron con el mismo compilador y los mismos parámetros:

```
tcc -o reporte_temperaturas original/reporte_temperaturas.c -lm  
tcc -o reporte_temperaturas_reconstruido  
reconstruido/reporte_temperaturas_reconstruido.c -lm
```

Los ejecutables generados no se incluyen en la entrega (restricción de la actividad); las evidencias de su ejecución y comparación se presentan a continuación.

5. Comparación

5.1 Comparación funcional (salida generada)

Métrica	Original	Reconstruido
SHA-256 de reporte.json	8a8f85ba7386...4b5	8a8f85ba7386...4b5 (idéntico)
Salida en consola	Mínimo: 19.8, Máximo: 33.1, Promedio: 26.81, Días de alerta: 2	idéntica

El archivo reporte.json generado por ambos ejecutables es byte a byte idéntico (mismo hash SHA-256), y el mensaje impreso en consola también coincide exactamente. Esto confirma que el comportamiento observable del programa se reconstruyó al 100%. Los archivos reporte.json de ambas ejecuciones se incluyen en las carpetas original/ y reconstruido/ de esta entrega.

5.2 Comparación binaria

Métrica	Original	Reconstruido
Tamaño	5224 bytes	5232 bytes
SHA-256	636e0115ec06...723	6b38a9c4e9cf...263
Formato	ELF 64-bit LSB, x86-64	ELF 64-bit LSB, x86-64
Bibliotecas enlazadas	libc.so.6, libm.so.6	libc.so.6, libm.so.6

Los binarios no son idénticos byte a byte (la primera diferencia aparece dentro del encabezado ELF/offsets), lo cual es el resultado esperado: TCC no genera binarios reproducibles bit a bit entre dos compilaciones distintas del mismo programa fuente, ya que cualquier variación mínima en el tamaño de los datos (en este caso, algunos bytes adicionales por comentarios del código reconstruido) desplaza las direcciones internas de las tablas de símbolos y reubicación. Un ejecutable funcionalmente equivalente no tiene por qué ser bit-idéntico.

5.3 Comparación a nivel de ensamblador

Se desensambló la función main de ambos binarios (objdump -d --disassemble=main) y se comparó instrucción por instrucción: mismo número de instrucciones, mismos opcodes y mnemónicos en cada posición. Las únicas diferencias corresponden a valores absolutos de direcciones (desplazadas por el cambio de tamaño en la sección de datos), no a la lógica del programa, que resultó ser exactamente la misma en ambas versiones.

5.4 Conclusión de la comparación

Criterio	¿Coincide?
Salida generada (JSON)	Sí — idéntica (mismo hash)
Salida en consola	Sí — idéntica
Lógica en ensamblador	Sí — idéntica
Tamaño de binario	Aproximadamente igual (diferencia de 8 bytes por offsets)
Binario byte a byte	No (esperado; no es objetivo del ejercicio)
Código fuente byte a byte	No (nombres de variables/comentarios difieren; la lógica no)

6. Conclusión general

La reconstrucción realizada con Ghidra (documentada en R003) fue funcionalmente exitosa: a partir de un binario compilado con TCC y sin símbolos de depuración, se recuperó código fuente en C válido que, al compilarse con el mismo compilador, produce un programa con comportamiento idéntico al original — mismo JSON de salida, mismo mensaje en consola, y lógica de ensamblador indistinguible salvo por desplazamientos de direcciones propios de cualquier recompilación.

Referencias

- TCC — Tiny C Compiler. <https://bellard.org/tcc/>
- Ghidra Software Reverse Engineering Framework — National Security Agency. <https://github.com/NationalSecurityAgency/ghidra>